

align(center + horizon)[block(width: 15cm, inset: (y: 1.1cm), stroke: (top: 2.5pt + rgb(26, 54, 93),
bottom: 0.75pt + rgb(203, 213, 225)),)[align(center)[text(size: 10.5pt, tracking: 0.35em, fill:
rgb(71, 85, 105))[大 作 业] v(0.55cm) text(size: 26pt, weight: “bold”, fill: rgb(15, 23, 42))[揭棋对弈
程序设计] v(0.65cm) text(size: 19pt, weight: “medium”, fill: rgb(30, 64, 175))[规则引擎设计]
v(0.35cm) if subtitle != “” [text(size: 13pt, fill: rgb(100, 116, 139))[Rule Engine Design — 坐标、棋
子、走法、终局] v(0.35cm)]]]

v(1.5cm)

box(width: 13cm, inset: (x: 1.2cm, y: 0.95cm), fill: rgb(248, 250, 252), radius: 6pt, stroke: 0.75pt +
rgb(226, 232, 240),)[align(center)[text(size: 11pt, weight: “bold”, fill: rgb(51, 65, 85))[小组成员]
v(0.55cm) grid(columns: (1fr, 1fr), column-gutter: 1.6cm, row-gutter: 0.65cm, align: center +
horizon, [text(size: 13pt, weight: “bold”)[张恒基 (组长)]
text(size: 12pt, fill: rgb(100, 116, 139))[2024211301 / 2024210926]], [text(size: 13pt, weight: “bold”)
[秦博宇]
text(size: 12pt, fill: rgb(100, 116, 139))[2024211302 / 2024210940]], [text(size: 13pt, weight: “bold”)
[陈艺博]
text(size: 12pt, fill: rgb(100, 116, 139))[2024211302 / 2024210931]], [text(size: 13pt, weight: “bold”)
[陈雨飞]
text(size: 12pt, fill: rgb(100, 116, 139))[2024211301 / 2024210918]],)]]

v(1cm) text(size: 12pt, fill: rgb(100, 116, 139))[技术设计 · 2026-06-18] v(0.5cm) text(size: 10.5pt,
fill: rgb(148, 163, 184))[北京邮电大学 · 计算机学院]]

设计目标

目标	说明	状态
服务端权威校验	所有走子经 Game.processMove 统一入口判定	已实现
明暗子分离	暗子按 virtualType 走子，明子按 type 走子	已实现
强化士象	明士可出九宫、明象可过河	已实现
终局统一出口	EndgameJudge.checkAfterMove 集中判定	已实现
错误原因细分	RuleValidator 仅返回 boolean	待强化

坐标系统

显示坐标（协议 / 用户界面）

列 a（左）→ i（右）；行 9（顶 / 黑方）→ 0（底 / 红方）。

	a	b	c	d	e	f	g	h	i	
	+---	+---	+---	+---	+---	+---	+---	+---	+---	
9										黑方底线
8										
7										
6										
5		----	----	----	----	----	----	----	----	河界
4										
3										
2										
1										
0										红方底线
	+---	+---	+---	+---	+---	+---	+---	+---	+---	

内部数组索引

概念	规则
显示行 → 内部行	row = 9 - displayRow
显示列 → 内部列	col = coord.charAt(0) - 'a'
棋盘尺寸	grid[10][9], row 0 = 顶行（黑方），row 9 = 底行（红方）
序列化	BOARD_STATE 中 row0 = 顶行，row9 = 底行

九宫范围

阵营	行范围（内部）	列范围
红帅	7-9	3-5（d-f 列）
黑将	0-2	3-5（d-f 列）

棋子模型

类型编码

编码	棋子	红方名	黑方名
0	KING	帅	将
1	ROOK	车	车

2	KNIGHT	马	马
3	CANNON	炮	炮
4	PAWN	兵	卒
5	ADVISOR	仕	士
6	BISHOP	相	象
-1	UNKNOWN	暗	暗

关键字段

字段	含义
type	真实身份；暗子未翻开时为 UNKNOWN
virtualType	该格开局原位对应的象棋角色；暗子按此走子
revealed	是否已翻开
color	RED(0) / BLACK(1)
row, col	当前格子坐标（内部索引）

getMoveType(): 若 revealed=false 返回 virtualType，否则返回 type。

明子与暗子示例

场景	type	virtualType	revealed	走子依据
明车	ROOK	ROOK	true	type = ROOK
暗车（原位为车）	UNKNOWN	ROOK	false	virtualType = ROOK
翻子后	服务器随机分配真实 type	不变	true	type（真实身份）

七种棋子走法规则

车 / 俥（ROOK）

- 同行或同列直线移动
- 路径上不得有棋子（吃子时可走到对方棋子格）
- 明暗差异：无；暗车与明车几何规则相同
- 边界：不得走出棋盘（0≤row≤9，0≤col≤8）

马 / 傴（KNIGHT）

- 移动：「日」字：(±2,±1) 或 (±1,±2)
- 蹩马腿：马前进方向相邻格有子则不可跳
- 明暗差异：无
- 典型拒例：马在 b7，b6 有子则不可到 d8 / a8

炮 / 砲（CANNON）

- 平移：同行或同列，路径无子
- 吃子：路径上恰好一个炮架，再吃目标格棋子
- 明暗差异：无
- 典型拒例：两子之间有两个阻隔时不能吃

兵 / 卒（PAWN）

- 未过河：只能向对方前进 1 格（红方向上 row 减小，黑方向下 row 增大）

- 已过河：红方 $\text{row} \leq 4$ 、黑方 $\text{row} \geq 5$ 视为过河；可前进 1 或左右平移 1
- 后退：禁止
- 明暗差异：无

将 / 帅 (KING)

- 移动：九宫内前后左右 1 格（曼哈顿距离 = 1）
- 照面：两将同列且中间无子时，走子后若仍照面则 isMoveLegal 拒绝
- 明暗差异：将帅开局即为明子

士 / 仕 (ADVISOR)

项目	暗子	明子（强化）
移动	斜走 1 格	斜走 1 格
活动范围	限九宫	全场（可出九宫）
判定依据	!revealed 时检查九宫	revealed 时仅检查斜 1 格

象 / 相 (BISHOP)

项目	暗子	明子（强化）
移动	田字（行差 2、列差 2）	同左
塞象眼	田字中心格有子则不可走	同左
过河	禁止（红 $\text{dst.row} \geq 5$ ，黑 $\text{dst.row} \leq 4$ ）	允许过河

暗子特殊规则

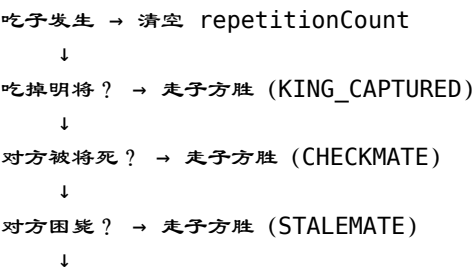
规则	说明	校验位置
走法按 virtualType	暗子不暴露真实身份，仅按原位角色生成走法	getMoveType()
暗士限九宫	与标准象棋士相同	isValidAdvisorMove
暗象不过河	与标准象棋象相同	isValidBishopMove
禁止原地翻子	source == destination 或 isFlipOnly 直接拒绝	Game.processMove
首次移动翻开	走子执行时 revealed ← true，type 由服务器随机	Board.executeMove
翻子不算吃子	无吃子目标的翻子仍递增 noCaptureCount	Board.executeMove

```
set text(size: 11pt, fill: rgb(3, 105, 161)) body
```

终局判定

终局入口：EndgameJudge.checkAfterMove，在 Board.executeMove 与棋谱记录之后调用。

判定优先级



noCaptureCount >= 80 ? → 和棋 (NO_CAPTURE_DRAW)

↓

更新 repetitionCount[boardHash]

↓

重复 >= 6 次 ? → 长将判负 / 兵卒长捉和 / 长捉判负

↓

继续对弈

各终局类型

将死 (CHECKMATE) : 对方无合法走法 且 正在被将军 → 走子方胜

困毙 (STALEMATE) : 对方无合法走法 且 未被将军 → 走子方胜

超时 (TIMEOUT) : 当前回合已用时间 > 65 000 ms (60 s 步时 + 5 s 裕量) → 超时方负

认输: 玩家主动发送 resign 或断线 → 对方胜

40 步无吃子和棋: noCaptureCount >= 80 (双方各 40 步无吃子) → GameState.DRAW

长将判负: 同一局面 (含行棋方) 重复 >= 6 次, 且当前步后仍在将军对方 → 将军方判负

长捉判负 / 兵卒长捉和: 重复 >= 6 次, 当前步未将军, 但走子后可合法吃某一对方子 (不含将) → 走子方判负 (兵卒则和棋)

重复局面哈希

局面键: Board.positionKey(board, sideToMove), 用于长将/长捉计数。发生吃子时 repetitionCount 整体清空。

校验流程

主流程

moveRequest 到达服务器

↓

对局状态 == PLAYING?

└ 否 → 拒绝: 对局未开始或已结束

└ 是 → 轮到你?

└ 否 → 拒绝: 还没轮到你

└ 是 → 已超时?

└ 是 → 终局: 超时方负

└ 否 → 原地翻子?

└ 是 → 拒绝: 禁止原地翻子

└ 否 → isValidMove 棋子规则

└ 否 → 拒绝: 非法走法

└ 是 → isMoveLegal 不送将

└ 否 → 拒绝: 不能送将

└ 是 → executeMove 执行走子

↓

record.append 棋谱

↓

EndgameJudge.checkAfterMove

↓

有终局 → 更新 status + 广播

gameOver

无终局 → 换手 + 广播 moveResult

两层校验职责

方法	职责	不检查
isValidMove	几何走法、阵营、吃己方、棋子类型规则	是否送将
isMoveLegal	试走后己方是否被将军	—
generateStrictLegalMoves	枚举全部不送将的合法走法	—

将军检测

isInCheck: 对方能否用合法几何走法吃到己方将帅（不考虑送将约束的 generateAllMoves + 目标为将）。

与网络层交互

阶段	领域层	网络层
接收走子	—	WsGameServer 解析 JSON → Move
校验执行	Game.processMove	—
翻子结果	Board 内部随机	moveResult.flipResult 广播
终局	GameStatus + gameOverReason	gameOver 消息
非法走子	返回错误字符串	error 消息，棋盘不变

测试覆盖

领域	测试类	状态
七种走法	BoardMakeMoveTest、DarkPieceRuleTest	已实现
送将 / 照面	RuleEdgeCaseTest	已实现
将死 / 困毙	GameEndgameTest、EndgameJudgeTest	已实现
40 步无吃子	EndgameJudgeTest	已实现
长将 / 长捉	EndgameJudgeTest、RuleEdgeCaseTest	待强化
undo / 搜索模拟	BoardUndoTest	已实现

已知限制

限制	说明	优先级
错误原因不细分	RuleValidator 返回 boolean, processMove 仅返回固定中文字符串	P1
长捉分类简化	「将 / 杀 / 捉」的精确象棋裁判分类未完全实现	P1
长捉边界	复杂连环捉、隔子捉等极端局面需人工复核	P2
客户端本地校验	客户端可走同一套 RuleValidator, 但与服务器版本必须同步	—
强化士象争议	已按课程/组内协议实现; 与传统揭棋规则可能不一致	—